Etude d'une cascade trophique

James Duburcq, Raphael Rideau, Orion Routhier

Avril 2026

Table des matières

1 Contexte de l'étude **2**

2 Résolution du Système d'EDO **2**
2.1 Méthode d'Euler implicite et Newton-Raphson 2
2.2 Méthode d'Euler Explicite 4
2.3 Méthode de Runge-Kutta d'ordre 5 4

3 Quelle méthode choisir ? **6**

4 Résulats simulation et interprétation **8**
4.1 Cas de base 8
4.2 Réintrodution des Loups 9
4.2.1 Cas de base 9
4.2.2 Paramètres permettant d'observer un équilibre 10

1 Contexte de l'étude

On se propose d'étudier une cascade trophique incluant cinq niveaux d'interaction : des végétaux, des ongulés principaux (orignaux/wapitis), des ongulés secondaires (cerfs), des loups et des ours. Afin de pouvoir étendre nos modélisations à n'importe quelle taille de réserve naturelle, nous considérerons la densité spatiale de ces populations. Celle-ci s'exprimera en nombre d'individus par kilomètre carré pour les animaux, et en kilogrammes par kilomètre carré pour la biomasse végétale.

Pour modéliser les dynamiques de ces populations, nous utiliserons un système d'équations différentielles qui dépasse les modèles classiques de proie-prédateur afin de capturer les véritables comportements écologiques observés sur le terrain. On obtient donc le système suivant :

$$\frac{dV}{dt} = \underbrace{r_V V\left(1-\frac{V}{K_V}\right)}_{\text{croissance logistique de la végétation}} - \underbrace{\frac{\alpha_N V N}{\nu_N+V}}_{\text{alimentation des wapitis/orignaux}} - \underbrace{\frac{\alpha_D V D}{\nu_D+V}}_{\text{alimentation des cerfs}} - \underbrace{\frac{\alpha_B V B}{\nu_B+V}}_{\text{alimentation des ours}} \tag{F1}$$

$$\frac{dN}{dt} = \underbrace{r_N N\left[1-\left(\frac{N}{\kappa_N(V)}\right)^{\theta_N}\right]}_{\text{croissance thêta-logistique}} - \underbrace{\frac{c_{WN} N W}{W+\beta N}}_{\text{prédation des loups}} - \underbrace{\frac{c_{BN} N B}{h_{BN}+N}}_{\text{prédation des ours}} - \underbrace{\delta_N N}_{\text{mortalité de fond}} \tag{F2}$$

$$\frac{dD}{dt} = \underbrace{r_D D\left[1-\left(\frac{D}{\kappa_D(V)}\right)^{\theta_D}\right]}_{\text{croissance thêta-logistique}} - \underbrace{\frac{c_{WD} D W}{h_{WD}+D}}_{\text{prédation des loups}} - \underbrace{\frac{c_{BD} D B}{h_{BD}+D}}_{\text{prédation des ours}} - \underbrace{\delta_D D}_{\text{mortalité de fond}} \tag{F3}$$

$$\frac{dW}{dt} = \underbrace{\left[\epsilon_1 \ln(\Phi_W+\eta)-\epsilon_2\right]W}_{\text{réponse numérique non linéaire}} - \underbrace{m_W W}_{\text{mortalité naturelle}} - \underbrace{\mu_W \sin^2(\pi t) W}_{\text{prélèvement humain saisonnier}} \tag{F4}$$

$$\frac{dB}{dt} = \left[e_B\left(\frac{c_{BN}N}{h_{BN}+N}+\frac{c_{BD}D}{h_{BD}+D}\right)+\frac{e_V \alpha_B V}{\nu_B+V}-m_B\right]B \tag{F5}$$

Et on prend le vecteur de conditions initiales :

$$u(0) = (V(0), N(0), D(0), W(0), B(0))^T = (440, 4.5, 2.5, 0.04, 0.25)^T$$

2 Résolution du Système d'EDO

2.1 Méthode d'Euler implicite et Newton-Raphson

On pose le vecteur d'état discrétisé au temps t_n :

$$y_n = (V_n, N_n, D_n, W_n, B_n)^T$$

tel que $(y_n)_n \in \mathbb{R}^5$.
On considère la méthode d'intégration d'Euler implicite, avec un pas de temps dt :

$$y_{n+1} = y_n + dt \times f(t_{n+1}, y_{n+1})$$

Avec la fonction vectorielle $f(t_{n+1}, y_{n+1})$ évaluée au temps futur t_{n+1} définie par le membre de droite de notre système d'EDO.

Comme cette équation est non-linéaire et que l'inconnue y_{n+1} apparaît des deux côtés de l'égalité, nous ne pouvons pas l'isoler directement. Le problème revient donc à trouver la racine de la fonction G définie par :

$$G(X) = X - y_n - dt \times f(t_{n+1}, X) = 0$$

Pour résoudre $G(X) = 0$, nous avons choisi d'implémenter la méthode itérative de Newton-Raphson. À partir d'une estimation initiale $X^{(0)} = y_n$, on calcule les termes suivants :

$$X^{(k+1)} = X^{(k)} - J_G(X^{(k)})^{-1} \times G(X^{(k)})$$

Où J_G est la matrice Jacobienne de la fonction G.

Afin d'appliquer cette méthode, nous avons d'abord développé une fonction permettant d'approximer numériquement la Jacobienne de notre système par différences finies :

```
def Jacobien(G, x, eps=10**(-6)):
    n = len(x)
    J = np.zeros((n,n))
    G0 = G(x)
    for j in range(n):
       x_var = x.copy()
       x_var[j] = x_var[j] + eps
       G_var = G(x_var)
       J[:,j] = (G_var - G0) / eps
    return J
```

Ensuite, nous avons implémenté le solveur basé sur la méthode de Newton-Raphson. À chaque itération, nous calculons la Jacobienne, puis en utilisant la décomposition LU avec matrice de permutation (code ci-après), nous résolvons le système linéaire associé pour mettre à jour notre estimation, jusqu'à atteindre une tolérance satisfaisante :

```

def decomposition_PA_LU(A):
    n = len(A)
    U = np.array(A, dtype=float)
    L = np.eye(n)
    P = np.eye(n)

    for k in range(n - 1):
        pivot_idx = np.argmax(np.abs(U[k:, k])) + k

        if pivot_idx != k:
            U[[k, pivot_idx], :] = U[[pivot_idx, k], :]
            P[[k, pivot_idx], :] = P[[pivot_idx, k], :]

            if k > 0:
                L[[k, pivot_idx], :k] = L[[pivot_idx, k], :k]

        for i in range(k + 1, n):
            if U[k, k] == 0:
                raise ZeroDivisionError("Pivot nul detecte.")

            alpha = U[i, k] / U[k, k]
            L[i, k] = alpha
            U[i, k:] = U[i, k:] - alpha * U[k, k:]

    return P, L, U
```

```
def Newton_Raphson(G, Un, Tsuivant, dt, maxiter=50, tol=1e-6):
    Uk = Un.copy()

    for k in range(maxiter):
        Gk = G(Uk)
        if(np.linalg.norm(Gk) <= tol):
            return Uk
        J = Jacobien(G, Uk)
        invJ_G = resoudre_PA_LU(J, Gk) # Resolution du systeme lineaire J*x = Gk
        Uk = Uk - invJ_G

    return Uk
```

Enfin, la résolution globale du système sur l'intervalle de temps se fait en construisant dynamiquement la fonction G à chaque pas de temps et en faisant appel à notre solveur :

```
def Euler_implicite(t, dt, U0, Fn, Maxiter=1000):
    interval_de_temps = [0]
    k = 0
    U_solution = [U0]

    while(k < Maxiter and interval_de_temps[-1] < t):
        Tsuivant = interval_de_temps[k] + dt
        interval_de_temps.append(Tsuivant)

        # Definition de la fonction G(X) = 0 a resoudre pour ce pas de temps
        def G(X):
            return X - U_solution[k] - dt * Fn(interval_de_temps[k+1], X)

        X = Newton_Raphson(G, U_solution[k], interval_de_temps[k+1], dt)
        U_solution.append(X.copy())
        k = k + 1

    return interval_de_temps, U_solution
```

2.2 Méthode d'Euler Explicite

Pour résoudre notre système d'EDO, nous pouvons aussi utiliser la méthode d'Euler explicite défini par :

$$y_{n+1} = y_n + dt \times f(t_n, y_n)$$

Pour cette méthode il n'y a pas besoins d'utiliser de méthode de résolution pour les équations non linéaires. Nous avons donc l'implémentation suivante :

```

def eulerexplicite(t, dt, U0, Fn , Maxiter= 1000):
    n = U0.shape[0]
    interval_de_temps=[0]
    k=0
    U_solution = [U0]
    while(k < Maxiter and interval_de_temps[-1]<t) :
        X = U0.copy()
        Tsuivant = interval_de_temps[k] + dt
        interval_de_temps.append(Tsuivant)
        for i in range(n):
            X[i] = U_solution[k][i] + dt * Fn(interval_de_temps[k], U_solution[k])[i]
        X
        U_solution.append(X.copy())
        k +=1
    return interval_de_temps , U_solution
```

2.3 Méthode de Runge-Kutta d'ordre 5

Une troisième méthode pour résoudre ce système d'EDO est une méthode de Runge-Kutta à 6 étapes (d'ordre 5).

On rappelle l'algorithme général de la méthode pour passer de l'instant t_n à l'instant $t_{n+1} = t_n + dt$:

$$t_{n,i} = t_n + c_i \times dt$$

$$p_{n,i} = f(t_{n,i}, y_{n,i})$$

$$y_{n,i} = y_n + dt \times \sum_{j=1}^{i-1} a_{ij} p_{n,j}$$

L'estimation finale au pas de temps suivant est donnée par une combinaison linéaire des pentes intermédiaires $p_{n,i}$:

$$y_{n+1} = y_n + dt \times \sum_{i=1}^{6} b_i p_{n,i}$$

Les coefficients c_i (abscisses pour le temps), a_{ij} (matrice des poids intermédiaires) et b_i (poids finaux de la combinaison linéaire) utilisés dans notre programme sont résumés dans le tableau de Butcher suivant :

$$\begin{array}{c|cccccc}
0 & 0 & & & & & \\
\frac{1}{4} & \frac{1}{4} & 0 & & & & \\
\frac{1}{4} & \frac{1}{8} & \frac{1}{8} & 0 & & & \\
\frac{1}{2} & 0 & -\frac{1}{2} & 1 & 0 & & \\
\frac{3}{4} & \frac{3}{16} & 0 & 0 & \frac{9}{16} & 0 & \\
1 & -\frac{3}{7} & \frac{2}{7} & \frac{12}{7} & -\frac{12}{7} & \frac{8}{7} & 0 \\
\hline
 & \frac{7}{90} & 0 & \frac{32}{90} & \frac{12}{90} & \frac{32}{90} & \frac{7}{90}
\end{array}$$

Dans notre implémentation en Python ci-dessous, les variables k_i correspondent aux pentes successives $p_{n,i}$. On retrouve explicitement les coefficients du tableau de Butcher dans le calcul de chaque étape :

```python
def rungekutta5(t, dt, U0, Fn, Maxiter=1000):
    n = U0.shape[0]
    interval_de_temps = [0]
    k = 0
    U_solution = [U0]

    while(k < Maxiter and interval_de_temps[-1] < t):
        Tsuivant = interval_de_temps[k] + dt
        interval_de_temps.append(Tsuivant)

        #Etape 1 : c_1=0
        k1 = Fn(interval_de_temps[k], U_solution[k])

        #Etape 2 : c_2=1/4, a_2,1=1/4
        k2 = Fn(interval_de_temps[k] + 1/4 * dt, U_solution[k] + dt * (1/4 * k1))

        #Etape 3 : c_3=1/4, a_3,1=1/8, a_32=1/8
        k3 = Fn(interval_de_temps[k] + 1/4 * dt, U_solution[k] + dt * (1/8 * k1 + 1/8
             * k2))
        #Etape 4 : c_4=1/2, a_4,2=-1/2, a_43=1
        k4 = Fn(interval_de_temps[k] + 1/2 * dt, U_solution[k] + dt * (-1/2 * k2 + 1
            * k3))

        #Etape 5 : c_5=3/4, a_5,1=3/16, a_54=9/16
        k5 = Fn(interval_de_temps[k] + 3/4 * dt, U_solution[k] + dt * (3/16 * k1 +
            9/16 * k4))

        #Etape 6 : c_6=1, a_6,1=-3/7, a_62=2/7, ...
        k6 = Fn(interval_de_temps[k] + 1 * dt, U_solution[k] + dt * (-3/7 * k1 + 2/7
            * k2 + 12/7 * k3 - 12/7 * k4 + 8/7 * k5))

        # Calcul de y_{n+1} final avec les poids b_i
        X = U_solution[k] + dt * (7/90 * k1 + 0 * k2 + 32/90 * k3 + 12/90 * k4 +
            32/90 * k5 + 7/90 * k6)

        U_solution.append(X.copy())
        k += 1

    return interval_de_temps, U_solution
```

3 Quelle méthode choisir ?

Comme le précise le sujet, le modèle écologique que nous étudions est un système raide. Cette raideur provient de la coexistence de processus se déroulant sur des échelles de temps très différentes : des dynamiques très rapides (comme la chute brutale d'une population due à la prédation ou la chasse saisonnière) et des dynamiques très lentes (comme le renouvellement démographique de la forêt ou des prédateurs sur plusieurs décennies).

Tout d'abord, nous avons mesuré le temps de calcul de nos méthodes :

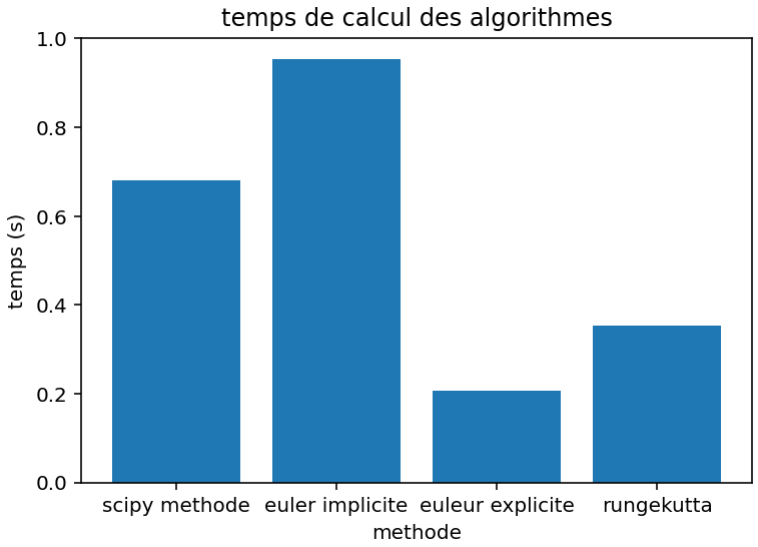


FIGURE 1 – Temps de calcul par méthode

Les premiers Résultats suggéreraient que la méthode d'Euler implicite serai la moins pertinente. En effet, celle ci est plus lente que les autres car elle nécessite de résoudre un système linéaire à chaque itération. Cependant, de part sa construction, cette méthode est plus robuste que les autres sur des systèmes raides.

Pour vérifier notre hypothèse, nous avons simulé un scénario extrême créant une pente initiale très abrupte : une forte surpopulation initiale de wapitis avec un pas de temps $dt = 0.02$. Les deux figures ci-dessous illustrent le comportement des méthodes face à cette instabilité initiale pour $N_0 = 23.0$ et $N_0 = 35.0$.

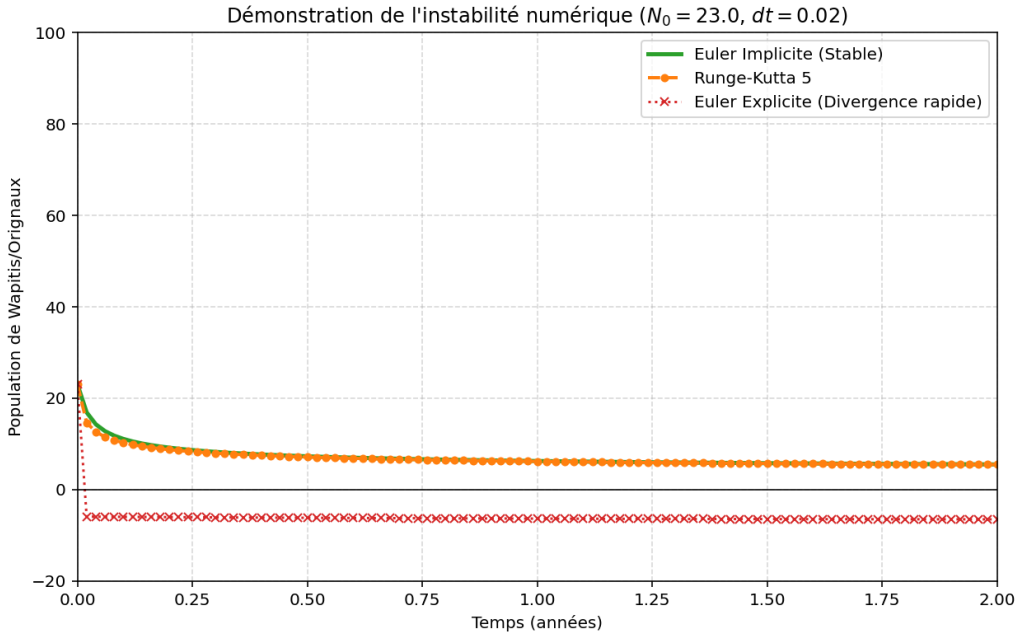


FIGURE 2 – Comparaison de la stabilité des méthodes pour une surpopulation modérée ($N_0 = 23.0$)

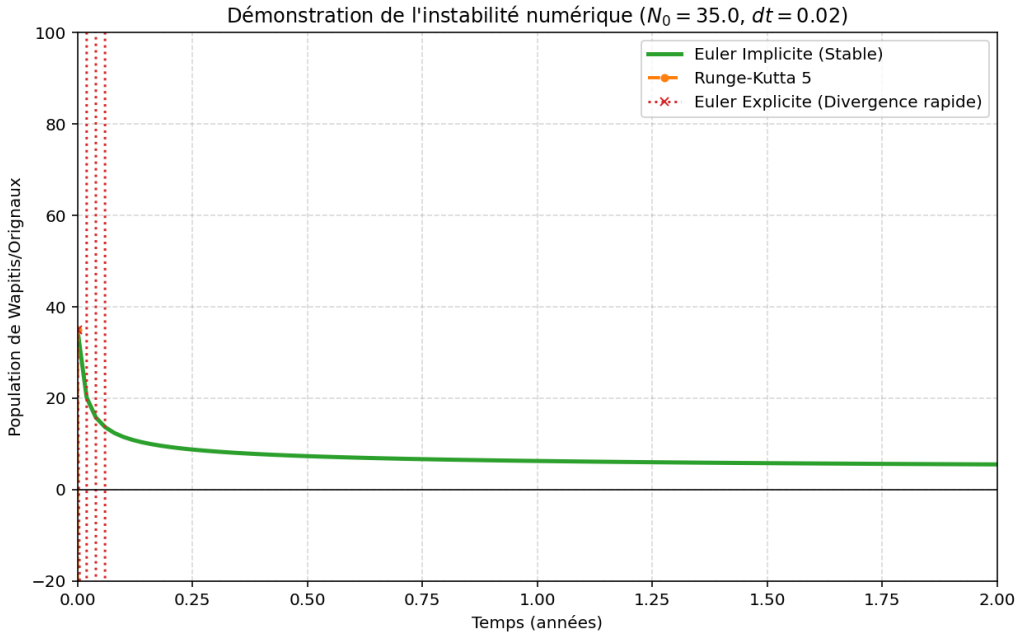


FIGURE 3 – Divergence des méthodes explicites face à une forte surpopulation ($N_0 = 35.0$)

Les méthodes explicites évaluent la pente au point actuel pour extrapoler le point suivant. Face à la dérivée initiale très fortement négative (due à une mortalité et une prédation intenses induites par la surpopulation), la méthode d'Euler Explicite surévalue cette décroissance et rate la courbe réelle. Dès le premier pas de temps, elle plonge dans des valeurs négatives (Figure 2) ou oscille frénétiquement jusqu'à exploser (Figure 3).
Bien que d'ordre supérieur, la méthode de Runge-Kutta 5 parvient à amortir le choc pour $N_0 = 23.0$ (Figure 1), mais finit par diverger elle aussi lorsque la contrainte devient trop forte à $N_0 = 35.0$ (Figure 2, disparition de la courbe).
Pour empêcher ces méthodes d'exploser, il faudrait réduire le pas de temps dt à des valeurs microscopiques, ce qui ferait exploser le temps de calcul pour une simulation sur 200 ans.

La solution : Euler Implicite :À l'inverse, la méthode d'Euler Implicite évalue la fonction au point d'arrivée (t_{n+1}).
Bien que cela nécessite de résoudre un système non linéaire à chaque étape (ce que nous faisons via Newton-Raphson), cette approche est stable. Comme on le voit sur les deux graphiques (courbe verte), l'Euler Implicite ne se laisse pas piéger par la pente abrupte : la méthode amortit naturellement le choc et ramène la population vers son équilibre de manière lisse et physiquement cohérente (sans jamais passer sous zéro).

Dans le cadre de notre étude sur des dizaines d'années, où les interactions proie-prédateur peuvent engendrer des variations de population très soudaines, il est impératif d'utiliser une méthode inconditionnellement stable. C'est pourquoi nous utiliserons exclusivement notre solveur basé sur la méthode d'Euler Implicite pour toutes les simulations écologiques de la suite de ce rapport.

4 Résulats simulation et interprétation

4.1 Cas de base

Afin de rendre le graphique plus lisible, nous avons décidé d'ajouter des coefficients multiplicatifs aux espèces étant les moins nombreuses. Ceci permet de mieux pouvoir apprécier les tendances sur une même échelle visuelle.

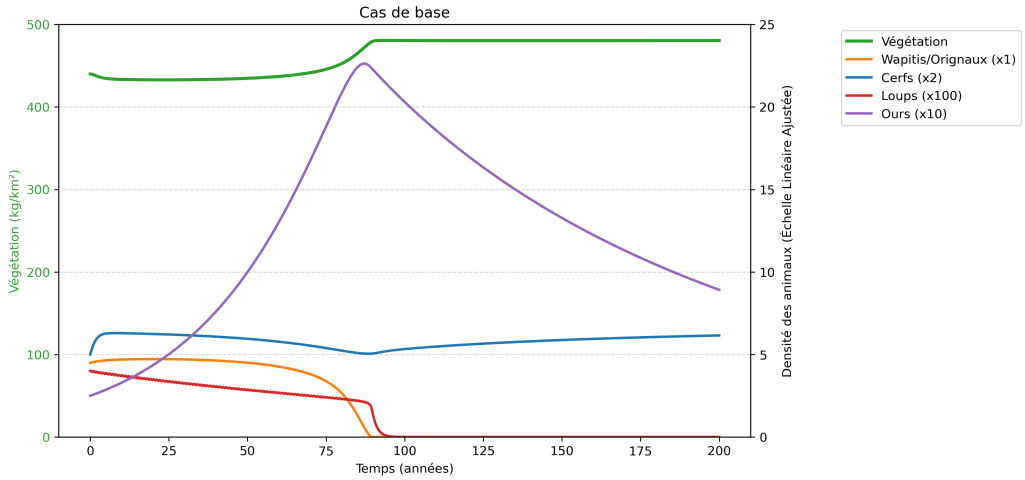


FIGURE 4 – Simulation du cas de base sur 200 ans avec les paramètres initiaux

L'observation de la simulation sur 200 ans avec les paramètres de base met en évidence un écosystème fondamentalement déséquilibré, menant à l'extinction de deux espèces (les wapitis et les loups) au profit d'un nouvel équilibre appauvri. L'analyse chronologique révèle trois phases distinctes :

- **Phase 1 : L'explosion de la population d'ours (Années 0 à 80).** Dès le début de la simulation, on observe une croissance exponentielle de la population d'ours (courbe violette). Biologiquement, les ours bénéficient de leur statut d'omnivores. Ils tirent leur énergie à la fois de l'abondante végétation initiale et de la prédation sur les deux espèces d'ongulés. N'ayant pas de prédateur naturel au-dessus d'eux et profitant de conditions optimales, leur population explose, créant une immense pression sur l'écosystème.
- **Phase 2 : Le déclin continu des loups et l'effondrement des wapitis (Années 80 à 100).** Contrairement à ce que l'on pourrait penser au premier abord, la population de loups (courbe rouge) décline lentement mais sûrement dès la première année. L'effondrement soudain des wapitis (courbe orange) vers l'année 85 n'est donc pas dû aux loups, mais à la surpopulation d'ours. Les loups subissent une forte mortalité naturelle combinée à la pression de la chasse humaine saisonnière ($\mu_W = 0.10$). Parallèlement, l'explosion démographique des ours exerce une pression de prédation insoutenable sur les wapitis. Les wapitis ayant un taux de croissance intrinsèque relativement faible ($r_N = 0.30$), ils ne parviennent pas à compenser les pertes et

s'effondrent. Cet effondrement donne le coup de grâce aux loups : privés de leur source de nourriture principale, ils s'éteignent définitivement. On observe ici un phénomène écologique de compétition interspécifique où la prolifération d'un prédateur omnivore (l'ours) évince le prédateur spécialiste (le loup).

— **Phase 3 : Le nouvel équilibre appauvri (Années 100 à 200).** Après la disparition des wapitis et des loups, le système se fige dans une nouvelle configuration :
  — **Végétation :** Les wapitis étant les plus gros consommateurs de biomasse végétale ($\alpha_N = 15.0$), leur disparition lève le contrôle descendant sur la flore. La végétation prolifère et sature le milieu pour atteindre sa capacité de charge maximale de 500 kg/km^2.
  — **Ours :** Privés de la viande de wapiti qui leur fournissait un fort gain énergétique, les ours voient leur population décliner lentement. Ils survivent néanmoins à un seuil plus bas grâce à leur régime omnivore composé de végétation et de cerfs.
  — **Cerfs :** La population de cerfs (courbe bleue) survit et se stabilise. Cela s'explique par leur taux de croissance intrinsèque plus élevé que celui des wapitis ($r_D = 0.50$ contre 0.30), ce qui leur confère une meilleure résilience face à la prédation résiduelle des ours.

4.2 Réintrodution des Loups

4.2.1 Cas de base

Tout d'abord, nous avons simulé la réintroduction de loup dans notre population en gardant les parametres de base donné dans le sujet.

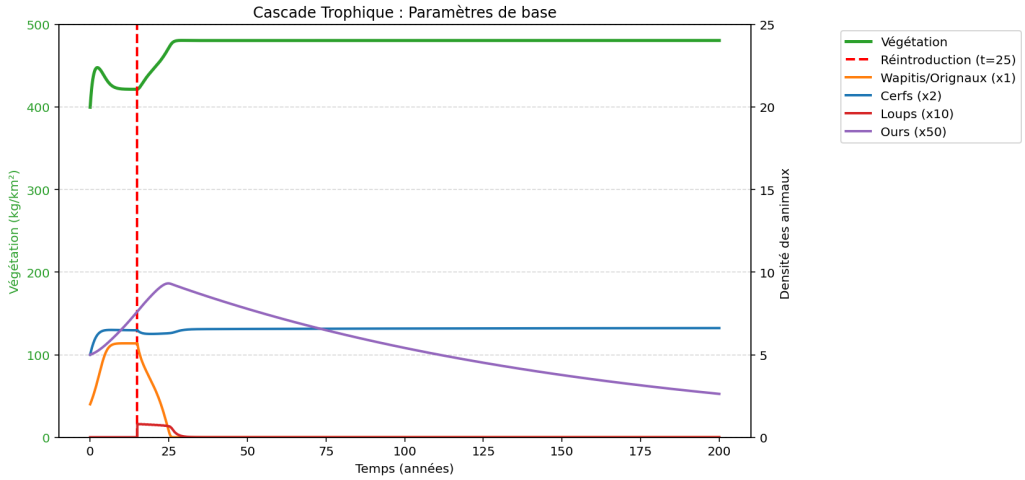


FIGURE 5 – Simulation du cas de base

L'observation de la simulation avec les paramètres de base met en évidence une instabilité écologique majeure : la réintroduction des loups provoque l'extinction rapide des wapitis, entraînant en cascade la disparition des prédateurs (loups et ours), et permettant à la végétation de saturer le milieu.
Ce phénomène s'explique par la très forte pression de prédation imposée par les paramètres initiaux. Le taux de mise à mort des loups sur les wapitis est extrêmement élevé ($c_{WN} = 7.5$) face à un taux de reproduction des wapitis relativement faible ($r_N = 0.30$).
Cette configuration simule un écosystème fermé et particulièrement vulnérable (similaire à l'écosystème insulaire d'Isle Royale mentionné dans l'étude). Dans un tel milieu confiné, les proies ne disposent pas de zones de refuge. Les prédateurs surexploitent donc leur ressource principale jusqu'à son épuisement total.
Sans wapitis, les loups meurent de faim, tout comme les ours qui perdent une part cruciale de leur apport énergétique. Libérée de la pression de pâturage ("contrôle ascendant"), la végétation prolifère jusqu'à atteindre sa capacité de charge maximale ($K_V = 500$).

4.2.2 Paramètres permettant d'observer un équilibre

Afin de pouvoir observer un équilibre suite à la réintroduction des loups dans le milieu, nous avons modifié les paramètres de la modélisation de manière empirique :

```

# 1. PARAMETRES DE LA V G TATION ET DES PROIES
r_V = 1.0          # Pousse de la vegetation
alpha_N = 20.0     # Appetit des wapitis
r_N = 0.60         # Les wapitis se reproduisent assez vite pour survivre
K_N = 15.0         # Capacite d'accueil confortable pour les wapitis
K_D = 6.0          # Capacite des cerfs

# 2. LE REGLAGE FIN DES LOUPS
c_WN = 2.0         # Efficacit  de chasse moderee
c_WD = 0.5         # Chasse sur les cerfs reduite
beta = 0.8         # Interference : les loups se genent legerement en chassant
epsilon_2 = 0.1    # M tabolisme ralenti : la meute survit avec moins de proies
m_W = 0.1          # Mortalite naturelle tres faible
mu_W = 0.03        # Pas de chasse humaine (indispensable pour un equilibre parfait)
r_D = 0.6          # Croissance des cerfs, au lieu de 0.3

# 3. CONTROLE DES OURS
e_V = 0.02
m_B = 0.11         # On augmente un peu leur mortalite pour qu'ils ne devorent pas tout
```

En réduisant le coefficient de chasse des loups (c_{WN} passant de 7.5 à 2.0), nous simulons un milieu où les wapitis ont la possibilité de fuir ou de se cacher, diminuant ainsi la probabilité de capture.
Le paramètre β a été augmenté de 0.5 à 0.8. Biologiquement, cela signifie que lorsque la meute de loups grandit, les individus se gênent mutuellement lors de la chasse, ce qui auto-régule l'efficacité de la prédation et empêche la surpopulation des prédateurs.
Le taux de reproduction des wapitis et des cerfs a été doublé (r_N et r_D à 0.6), ce qui leur permet de compenser les pertes dues à la prédation continue.
Dans cette configuration, nous observons une véritable "cascade trophique" réussie : les loups contrôlent la population de wapitis sans l'anéantir. Ce contrôle empêche les ongulés de surpâturer, ce qui maintient la végétation à un niveau sain (autour de 200 à 300 kg/km^2), garantissant ainsi la survie à long terme de toutes les strates de l'écosystème. La suppression quasi-totale du prélèvement humain ($\mu_W = 0.03$) permet par ailleurs à la dynamique naturelle de s'installer sans forçage externe perturbateur.

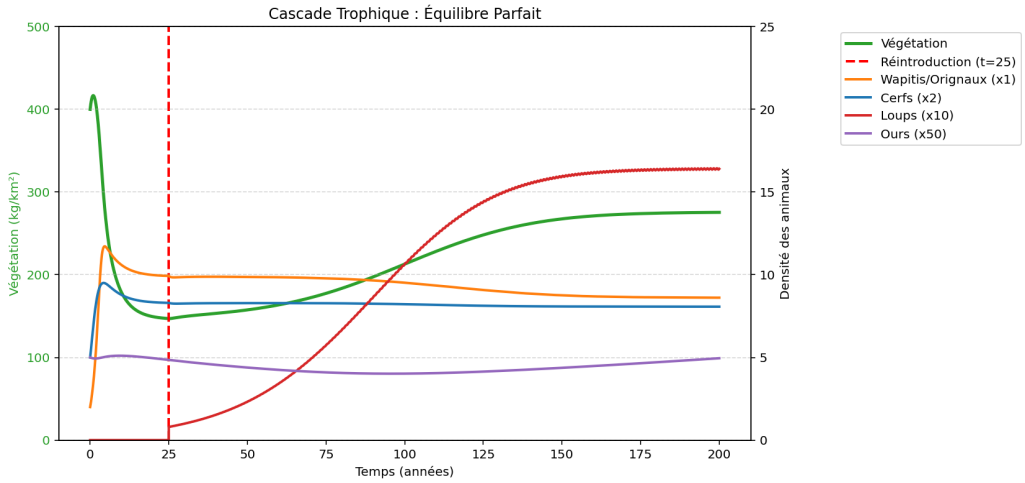


FIGURE 6 – Simulation réintroduction de loups avec paramètres ajustés